Software Documentation

PARMCO Project — Checkpoint 2
AI-Generated Documentation
Subsystem: RP4 control software (C and ARM assembly)

Table of Contents

1. Overview 1
2. Software Architecture 1
3. Library Attribution 2
4. How Each Library Works 3
4.1 Class library E4235 (instructor-provided) 3
4.2 BlueZ and RFCOMM 3
4.3 POSIX threads 3
4.4 termios and non-blocking I/O 3
4.5 time.h 3
5. Code Walkthrough 4
5.1 Keyboard commands (main thread) 4
5.2 Functions 4
5.3 Timing and RPM formulas 4
6. Build and Run 5
7. Code Flags (Human-Edit Markers) 5

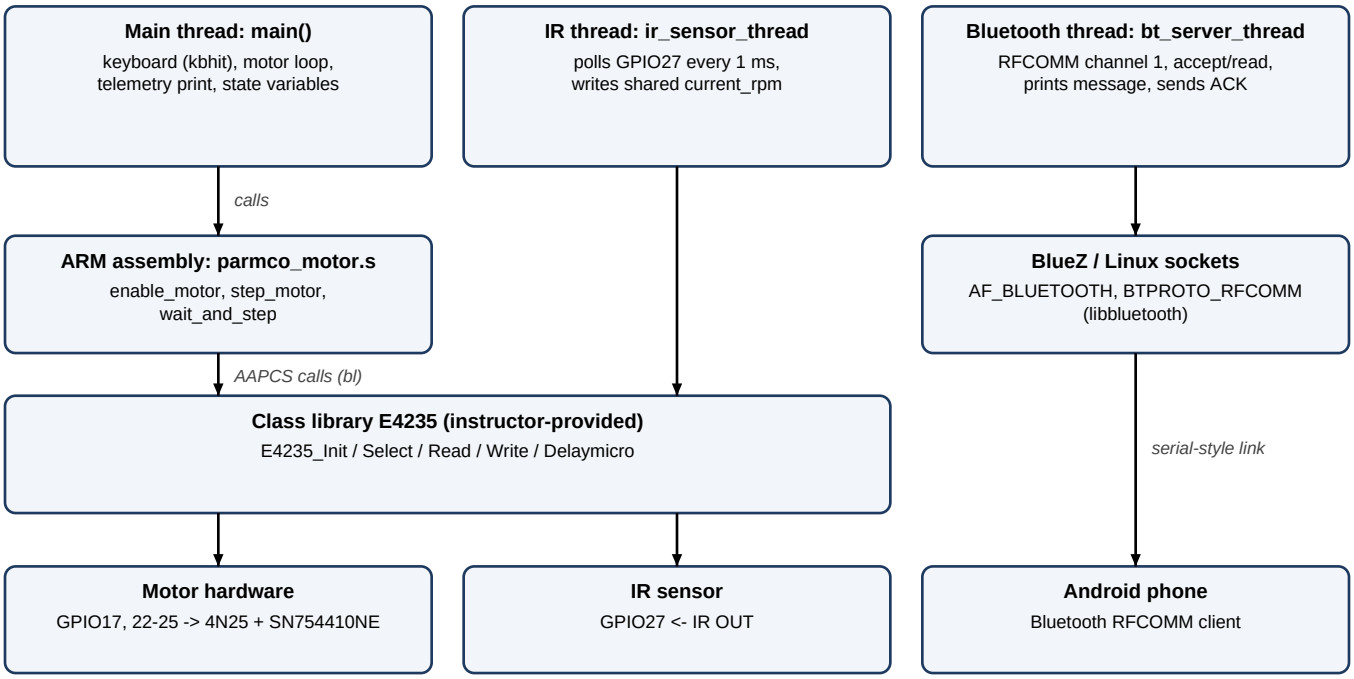
1. Overview

This document describes the Checkpoint 2 RP4 software: `main.c` (C: keyboard control, IR RPM measurement, Bluetooth RFCOMM server) and `parmco_motor.s` (ARMv7 AArch32 assembly: H-bridge enable routine, step routine, step-timing wrapper). Both were generated by an AI assistant; the class RP4 library (`E4235`) is provided by the instructor and is not AI-written. At Checkpoint 2 the motor is controlled only from the keyboard. The phone connection exchanges messages with the RP4, which are printed and acknowledged but do not move the motor.

Hardware connections used by this code (BCM GPIO numbers): GPIO17 motor enable (via the 4N25), GPIO22, 23, 24, 25 coil inputs 1A, 2A, 3A, 4A of the SN754410NE, and GPIO27 IR sensor input. See the Checkpoint 2 hardware documentation for the circuit.

2. Software Architecture

The program runs three threads. The main thread owns the keyboard, all motor state and the step loop. The IR thread measures propeller RPM. The Bluetooth thread serves the phone. After initialization, the motor's enable and step signals reach the hardware only through the assembly routines, and all GPIO access goes through the class library.



3. Library Attribution

Every library the code uses is listed here with its source, what it provides, and why the project needs it.

Library / header	Provider (attribution)	Used for in this code	Why it is needed
E4235 class library (`E4235.h`)	ECSE 4235 instructor. Not written by AI; the AI uses it.	E4235_Init, E4235_Select, E4235_Read, E4235_Write, E4235_Delaymicro	Project rule: all GPIO access, from C or assembly, goes through the class library. Assembly never touches GPIO registers directly.
BlueZ (`bluetooth/bluetooth.h`, `bluetooth/rfcomm.h`, libbluetooth)	BlueZ project, the official Linux Bluetooth stack (bluez.org). Open source.	sockaddr_rc, AF_BLUETOOTH, BTPROTO_RFCOMM, BDADDR_ANY	Gives C a serial-style Bluetooth link to the Android phone.
POSIX threads (`pthread.h`)	POSIX threads in the GNU C Library (glibc) on Raspberry Pi OS.	pthread_create, pthread_join	Runs IR sensing and the Bluetooth server beside the motor loop so none of them blocks the others.
POSIX terminal I/O (`termios.h`)	glibc / POSIX.	tcgetattr, tcsetattr, ICANON, ECHO, VMIN, VTIME	Reads single key presses with no Enter key and no echo, without blocking.
POSIX/Linux system calls (`unistd.h`, `fcntl.h`, `sys/socket.h`)	glibc / Linux kernel.	read, write, close, usleep, fcntl (O_NONBLOCK), socket, bind, listen, accept	Keyboard and socket I/O, non-blocking mode, and sleeping between polls.
C time (`time.h`)	glibc.	clock_gettime(CLOCK_MONOTONIC)	A clock that cannot jump, for IR pulse spacing, the debounce, and the once-per-second telemetry.

Library / header	Provider (attribution)	Used for in this code	Why it is needed
C standard library (`stdio.h`, `stdlib.h`, `string.h`)	glibc.	printf, perror, atexit, memset, memcpy, strcspn, strlen	Terminal output, buffer handling, and restoring the terminal on exit.

4. How Each Library Works

4.1 Class library E4235 (instructor-provided)

This section describes the library interface as the code uses it. The call pattern is: `E4235_Init()` once at start-up; `E4235_Select(pin, mode)` sets a pin as an output (mode 1) or input (mode 0); `E4235_Write(pin, value)` drives an output to 0 or 1; `E4235_Read(pin)` returns the level of an input; `E4235_Delaymicro(us)` waits for the given number of microseconds. Pin numbers are BCM GPIO numbers (the code notes that GPIO27 is header pin 13). The assembly calls `E4235_Write` and `E4235_Delaymicro` with `bl` using the ARM calling convention (AAPCS): first argument in r0, second in r1.

4.2 BlueZ and RFCOMM

RFCOMM emulates a serial port over a Bluetooth link. BlueZ exposes it to C as an ordinary stream socket, so the server uses the same call order as a TCP server: `socket(AF_BLUETOOTH, SOCK_STREAM, BTPROTO_RFCOMM)`, `bind` to a `sockaddr_rc` (any local adapter, channel 1), `listen`, then `accept`. After a phone connects, `read` returns the bytes it sent and `write` sends bytes back. Pairing and remembering the connection are done by the BlueZ background service and the phone, not by this program; BlueZ stores the pairing keys on the RP4. The code does not register a service record (SDP), so the phone app must connect to RFCOMM channel 1 directly, or a Serial Port service record must be registered separately.

4.3 POSIX threads

`pthread_create` starts a function in a new thread that shares the program's memory; `pthread_join` waits for it to finish. The threads share `app_running` (written only by the main thread) and `current_rpm` (written only by the IR thread). There is no mutex: each variable has a single writer and holds one 32-bit value, which is adequate here. The `volatile` keyword makes each thread re-read the variable instead of caching it. On quit the main thread clears `app_running` and joins both threads.

4.4 termios and non-blocking I/O

`set_nonblocking_terminal` saves the current terminal settings, registers `atexit` to restore them, then turns off ICANON (line buffering) and ECHO and sets VMIN=0, VTIME=0 so `read` returns immediately with 0 bytes when no key is pressed. `kbhit` uses that to return a key or -1. The two Bluetooth sockets use `fcntl(..., O_NONBLOCK)` so `accept` and `read` never block, which lets the thread see `app_running` fall and exit when Q is pressed.

4.5 time.h

`clock_gettime(CLOCK_MONOTONIC, ...)` returns seconds and nanoseconds since an arbitrary start that never goes backwards. The code subtracts two readings to get elapsed seconds.

5. Code Walkthrough

5.1 Keyboard commands (main thread)

Key	Action	State changed
E	Enable / disable the motor (GPIO17 through the 4N25)	motor_enabled, enable_motor()
D	Toggle direction CW / CCW	direction (+1 / -1)
R	One revolution: 400 half steps in the selected direction, then stops	steps_remaining = 400, is_running = 0
S	Start / stop continuous run	is_running
+ (or =)	Faster: step rate x 1.05, printed after each press	step_rate
- (or _)	Slower: step rate / 1.05, printed after each press	step_rate
Q	Disable the motor and shut down all threads	app_running = 0

Defaults: step_rate 133.3 steps/s (20 RPM in half-step mode), direction clockwise, motor disabled and stopped. The motor steps only while it is enabled and either continuous run is on or steps remain.

5.2 Functions

Function	File	What it does
main	main.c	Initializes the library, sets GPIO17, 22-25 as outputs, calls enable_motor(0), starts the two threads, prints the start-up banner (half-step mode and key list), then loops: read key, print telemetry once per second while running, and step the motor.
ir_sensor_thread	main.c	Sets GPIO27 as input and polls it every 1 ms. On a low-to-high change it measures the time since the previous pulse (ignoring pulses closer than 0.05 s) and sets current_rpm = 30 / seconds. If no pulse arrives for 2 s it sets current_rpm to 0.
bt_server_thread	main.c	RFCOMM server on channel 1. Accepts one phone at a time, reads a message, strips the line ending, prints it, and writes back "ACK: Message received". A read of 0 bytes means the phone disconnected, and the thread returns to waiting for a new connection.
kbhit, set_nonblocking_terminal, reset_terminal_mode	main.c	Non-blocking single-key input; the terminal is restored at exit.
enable_motor(state)	parmco_motor.s	Moves state into r1, loads 17 into r0, calls E4235_Write(17, state).
step_motor(direction, mode)	parmco_motor.s	Adds direction to current_step_index, wraps it with AND 7, looks up the byte in half_step_table (0x08, 0x0C, 0x04, 0x06, 0x02, 0x03, 0x01, 0x09), and writes its four bits to GPIO22 (bit 3), GPIO23 (bit 2), GPIO24 (bit 1), GPIO25 (bit 0). The mode argument is accepted but ignored (half-step only).
wait_and_step(delay_us, direction)	parmco_motor.s	Calls E4235_Delaymicro(delay_us), then step_motor(direction, 0). The main loop calls it with delay_us = 1,000,000 / step_rate.

5.3 Timing and RPM formulas

Half-step mode has 400 steps per revolution, so commanded RPM = step_rate x 60 / 400 = step_rate x 0.15 (133.3 steps/s = 20 RPM, 666.7 steps/s = 100 RPM). The propeller has two blades, so consecutive IR pulses are half a revolution apart and measured RPM = 60 / (2 x seconds between pulses) = 30 / seconds. Each step costs the delay plus the time of four GPIO writes, so the real step rate is slightly below the target, mostly at high speed.

6. Build and Run

The code needs the BlueZ development headers (on Raspberry Pi OS, the package `libbluetooth-dev`), the class library supplied by the instructor, and a 32-bit ARM toolchain (the assembly is ARMv7 AArch32). The two source files are compiled and linked together with the Bluetooth and thread libraries, and with the class library in the way the course instructions specify:

```
gcc -o parmco main.c parmco_motor.s -lbluetooth -lpthread
```

For the phone link, the RP4's Bluetooth adapter must be powered, pairable and visible to the phone.

7. Code Flags (Human-Edit Markers)

The ground rules require every human change in code to be marked. Each change is opened with a `[HUMAN EDIT START]` comment that gives the reason, keeps the AI original as a comment beginning `AI original:`, and is closed with an `[END HUMAN EDIT]` comment. Both source files were scanned for these markers and none were found, so both files are the unmodified AI original.

File	Human edits
main.c and parmco_motor.s	None